UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

INFORMATICS
Universitas
Multimedia
Nusantara

# IF351L - M06

# DATABASE SYSTEMS

## SELECT STATEMENT 01

MAHASISWA MAMPU MENGGUNAKAN SELECT STATEMENT (C3)



ivander.nathanael@student.umn.ac.id



# REVIEW SESI LECTURE MEETING 05

# INTRO TO SELECT STATEMENT & SIMPLEST SELECT STATEMENT

- **Sedikit Review Materi Kemarin**

Kemarin kita sudah membahas DDL, DML, terus bagaimana cara membuat, menampilkan, dan menghapus DB dan table, tipe data kolom, populating table, dll.

- **Materi Hari Ini**

Hari ini kita akan membahas SELECT & FROM, sisanya diminggu depan (Cuma karena quiz ya nanti saya kasih duluan materinya).

- **Simplest Select Statement**

***SELECT 67 * 67;***

  - Statement terkecil hanya berisi klausa SELECT tanpa FROM.

```
SELECT      ...
FROM        ...
WHERE       ...
GROUP BY    ...
HAVING      ...
ORDER BY    ...
LIMIT       ...;
```

```
MariaDB [(none)]> SELECT 67*67;
+-------+
| 67*67 |
+-------+
|  4489 |
+-------+
1 row in set (0.001 sec)
```

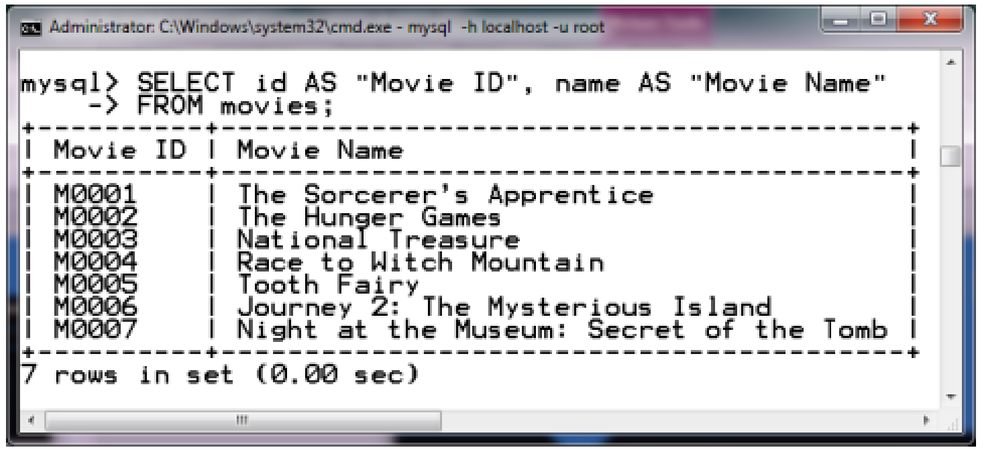
# ASSIGNING NAMES TO RESULT COLUMNS

Kita dapat memberikan nama pada kolom hasil dengan klausa “AS”, ini bersifat opsional.

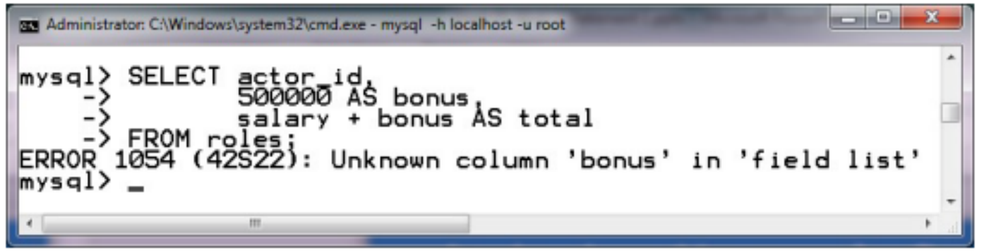
SYNTAX: ***SELECT column_name AS alternative_column_name, . . . FROM table_name;***

- Kalau Aliasnya ada spasi, harus diberi tanda kutip, supaya bisa dianggap sebagai 1 string.
- Alias tidak bisa dipakai di klausa SELECT yang sama.

```
SELECT id AS “Movie ID”, name AS “Movie Name”
FROM movies;
```

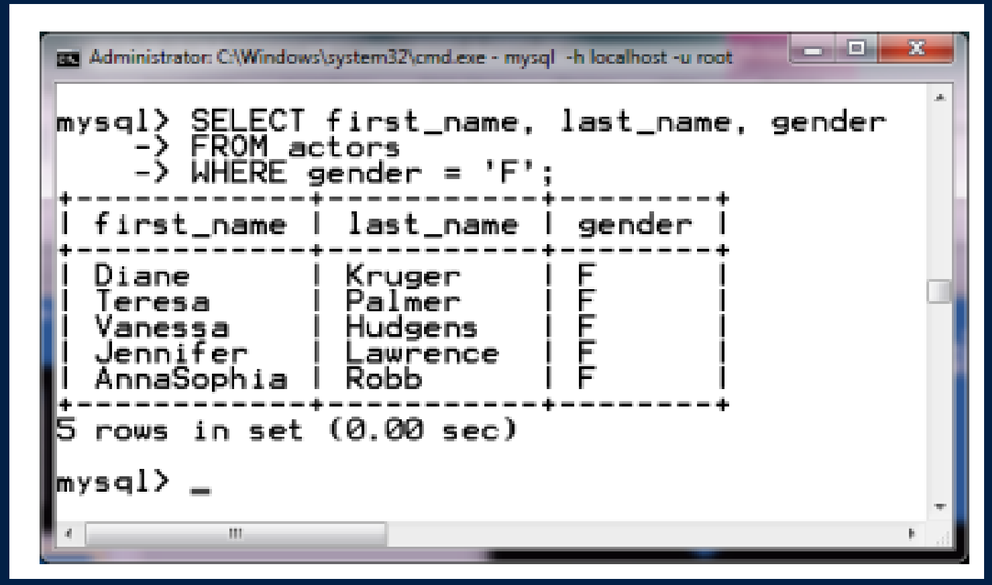


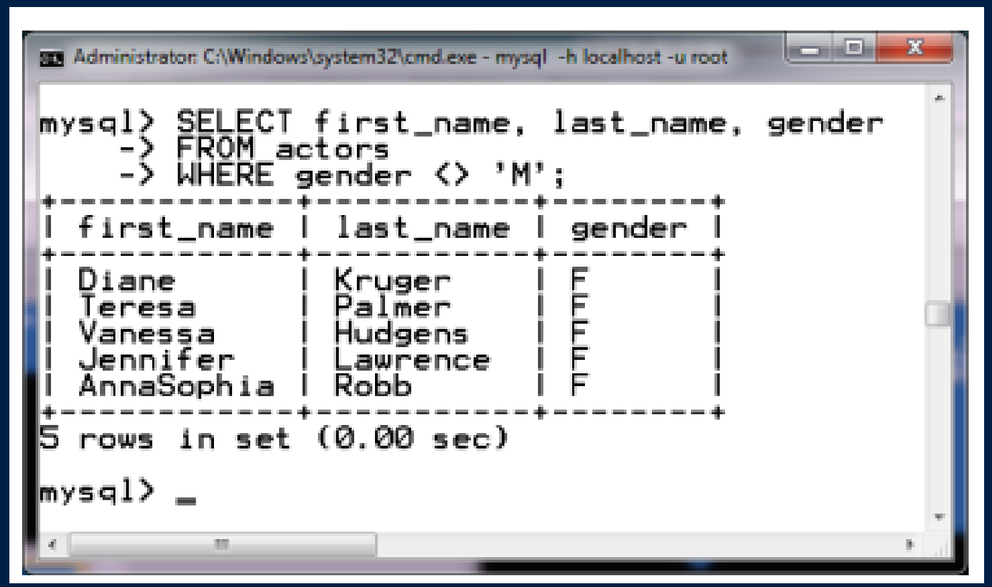

```
SELECT actor_id,
       500000 AS bonus,
       salary + bonus AS total
FROM roles;
```



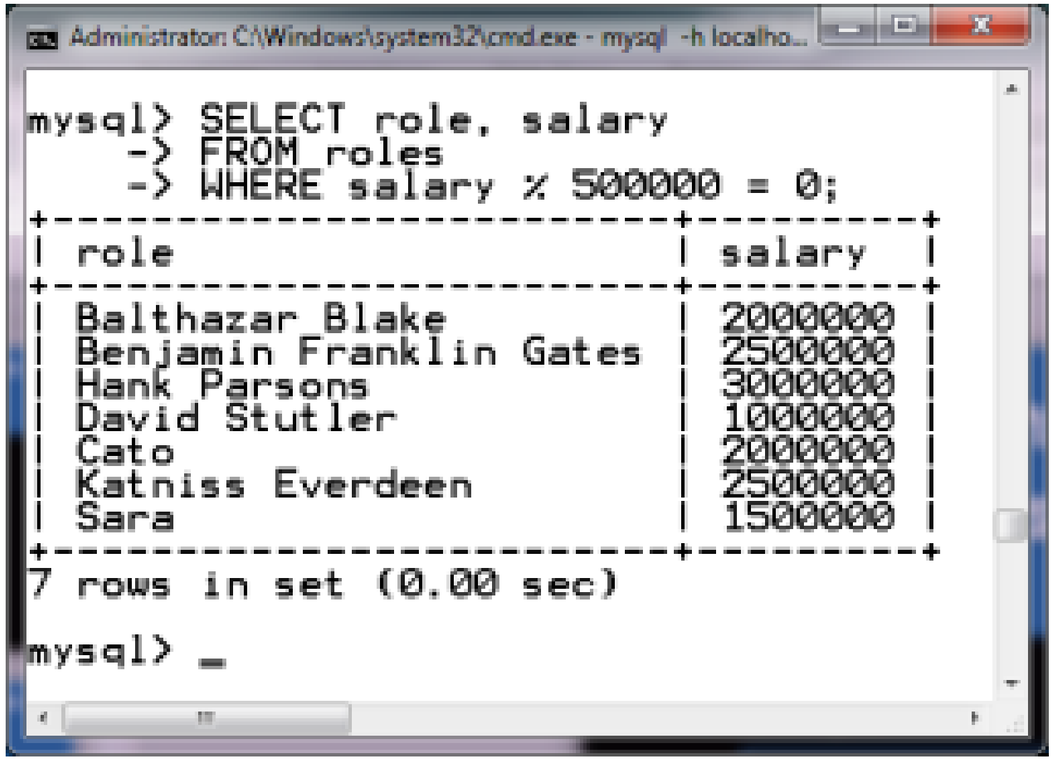
# CONDITIONS USING COMPARISON OPERATORS

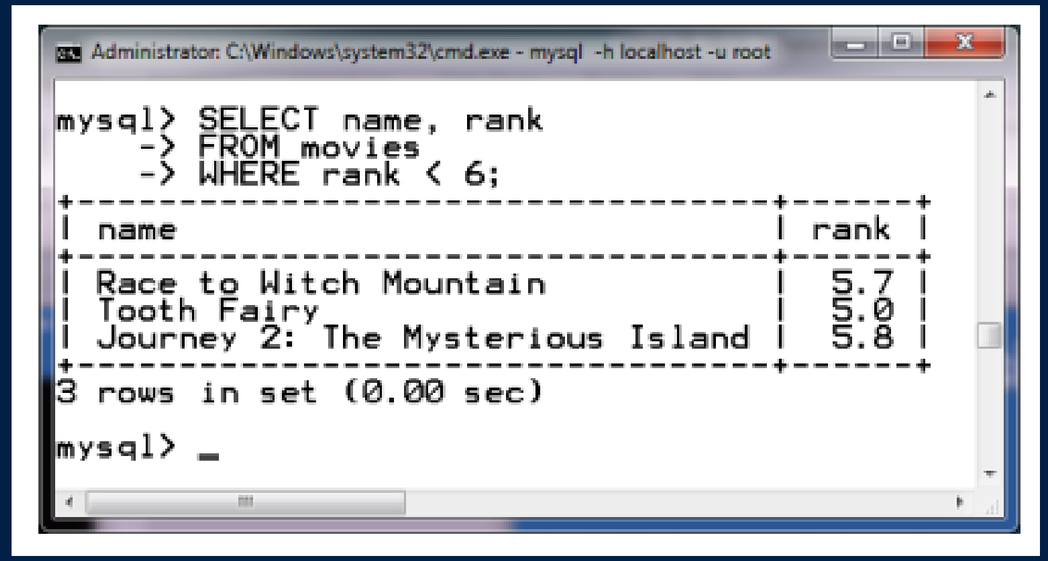
| OPERATOR | MEANING |
|---|---|
| = | Equal To |
| <=> | NULL Safe Equal |
| < ; > ; <= ; >= | Less Than, .... |
| <> ; != | Not Equal To |

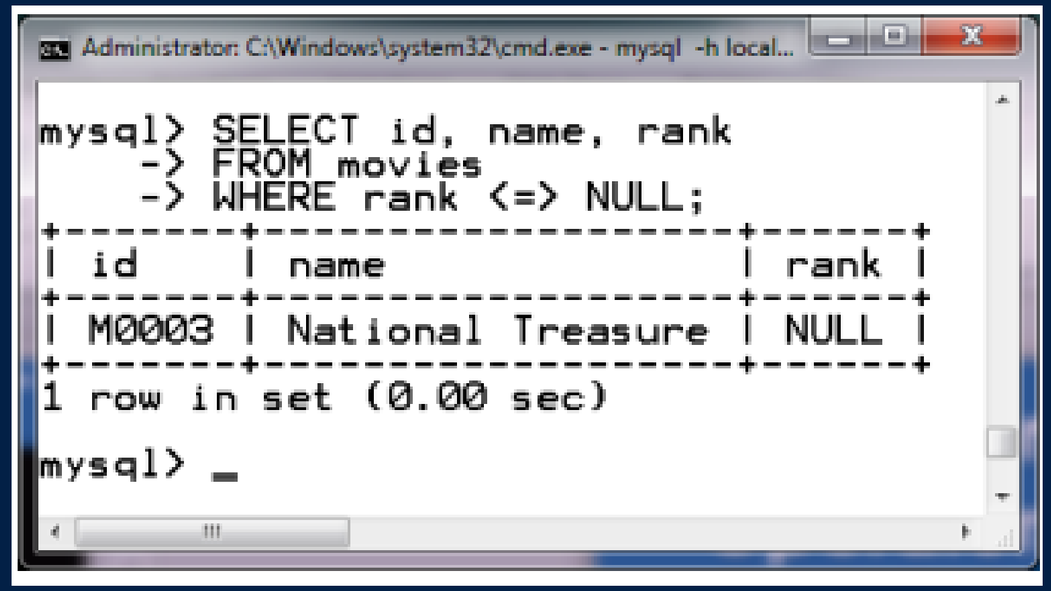


# CONDITIONS USING COMPARISON OPERATORS



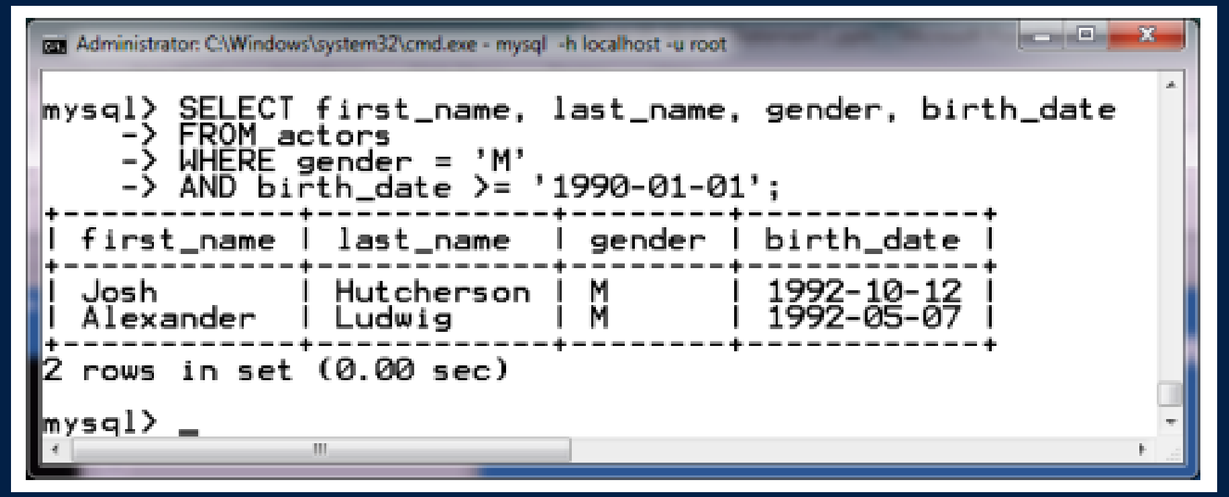



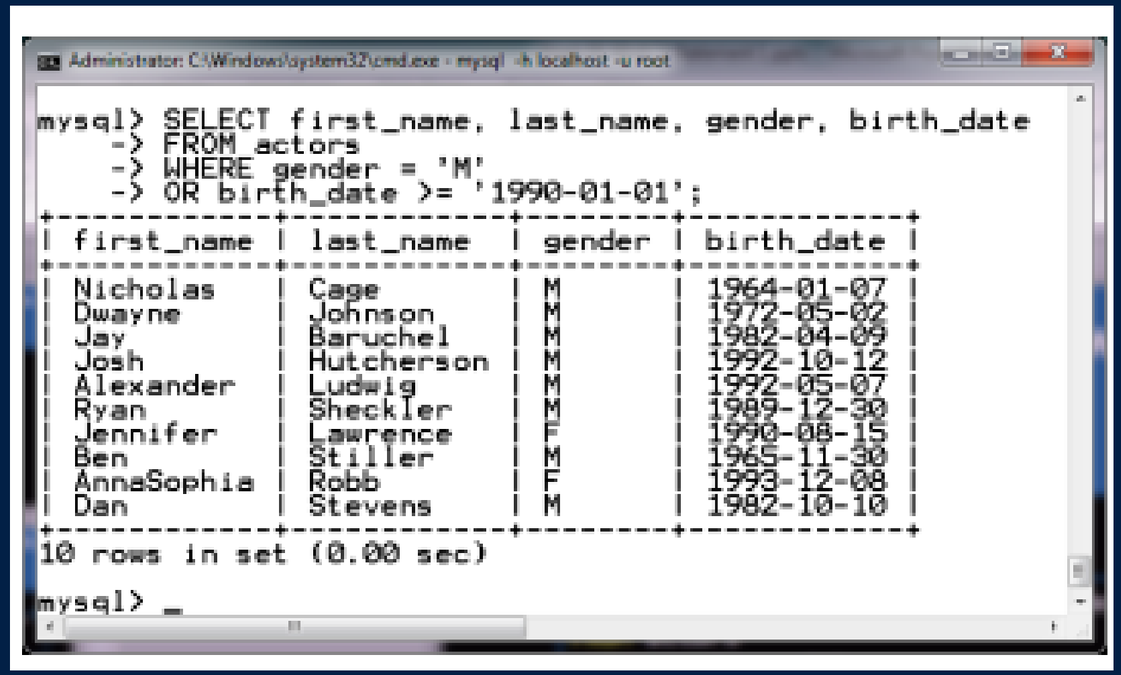



# CONDITIONS COUPLED WITH AND, OR, XOR, & NOT

Kita dapat menggunakan Logical Operator AND, OR, XOR, ataupun NOT dalam melakukan filtering:

- A AND B → Keduanya benar
- A OR B → Salah satu atau keduanya benar
- A XOR B → Tepat salah satu yang benar
- NOT → Negasi

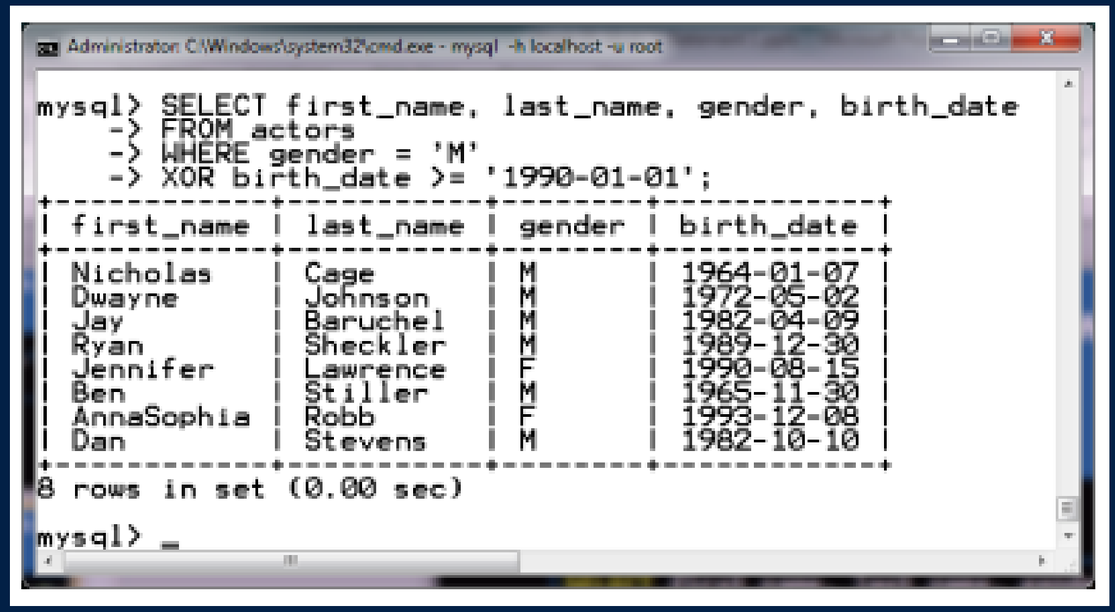


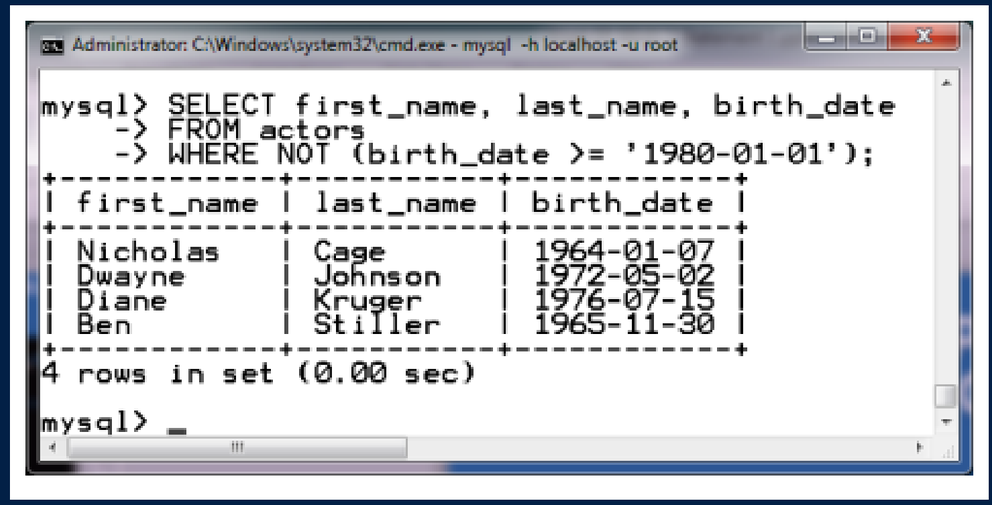



# CONDITIONS COUPLED WITH AND, OR, XOR, & NOT

Kita dapat menggunakan Logical Operator AND, OR, XOR, ataupun NOT dalam melakukan filtering:

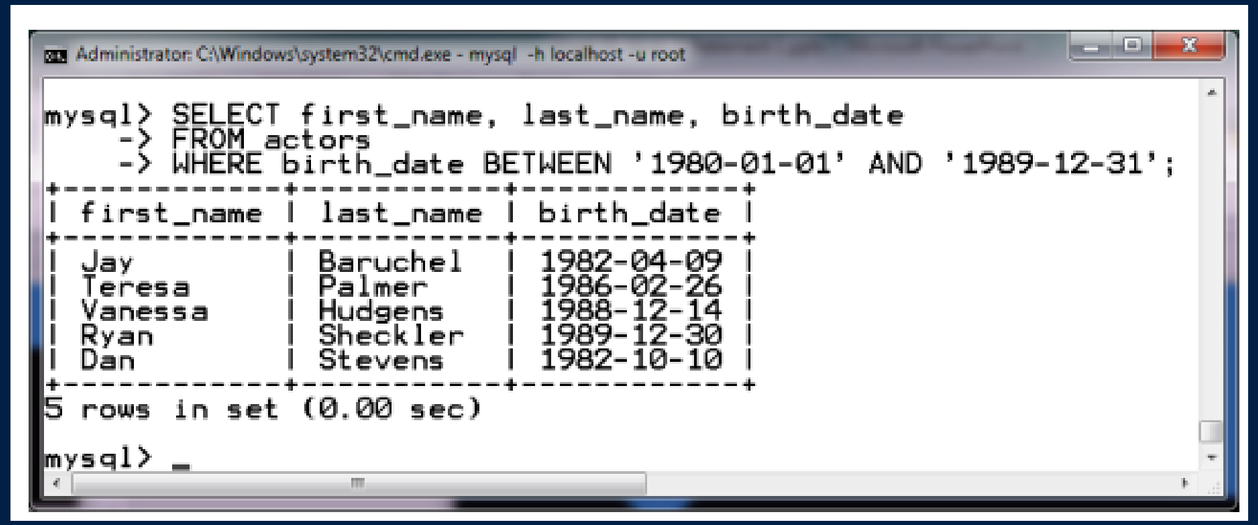
- A AND B → Keduanya benar
- A OR B → Salah satu atau keduanya benar
- A XOR B → Tepat salah satu yang benar
- NOT → Negasi



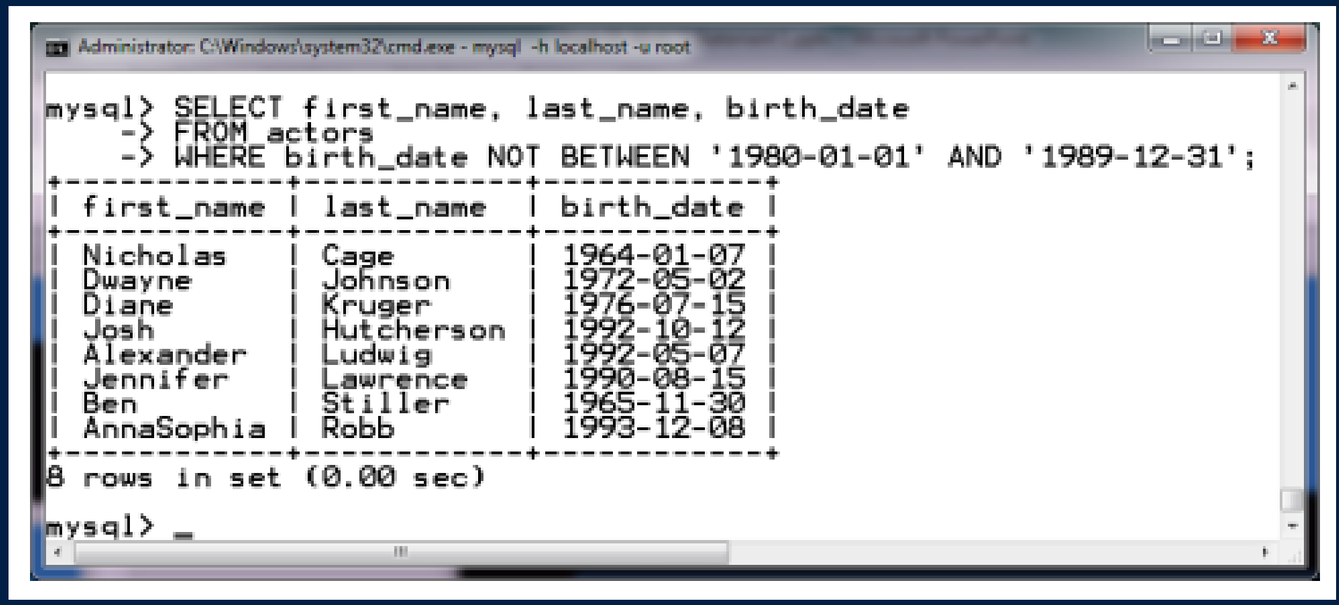
# BETWEEN OPERATOR

Kita dapat menggunakan BETWEEN untuk mencari entry dengan range tertentu. Dimana Batas atas dan bawah sudah termasuk karena inklusif. Ada juga kebalikannya yaitu NOT BETWEEN.




Syntax:

- x BETWEEN ‘a’ AND ‘b’
- x NOT BETWEEN ‘a’ AND ‘b’

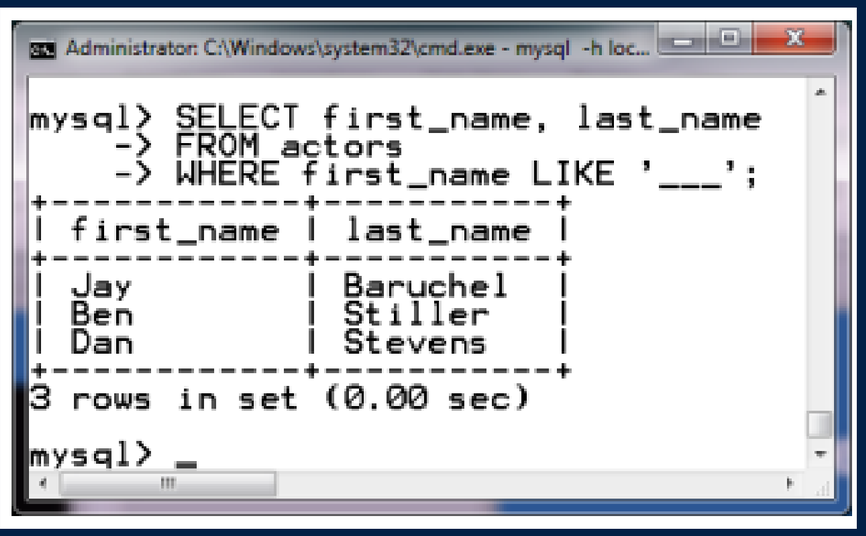


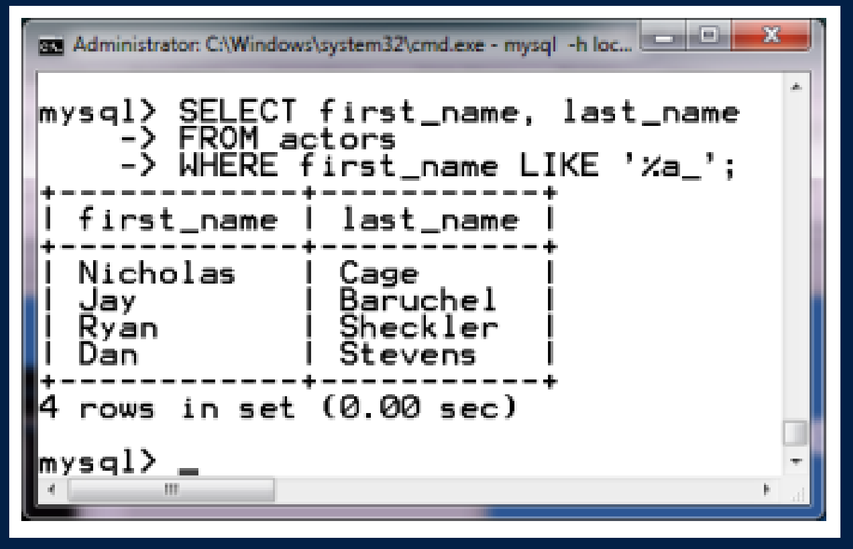
# LIKE OPERATOR

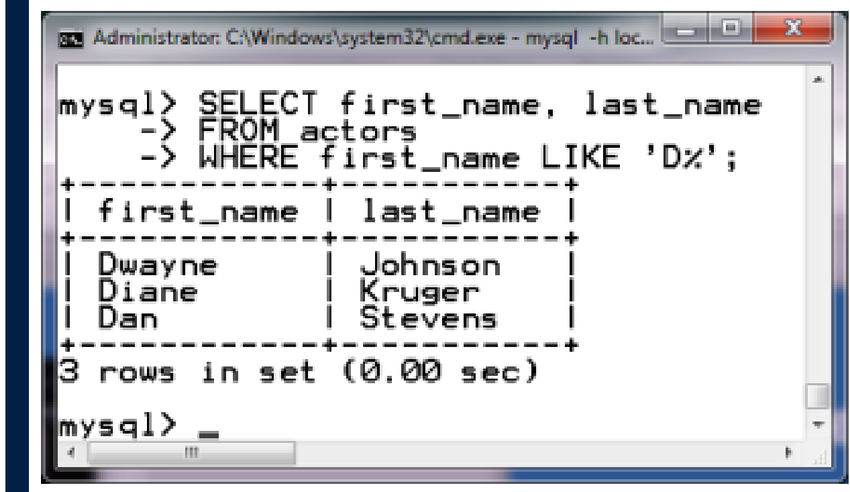
LIKE Operator biasanya dipakai untuk REGEX atau Pencocokan Pola.

- % cocok dengan 0 karakter atau lebih
- _ cocok dengan tepat 1 karakter.

| POLA | ARTI | HASIL |
|---|---|---|
| 'D%' | Diawali Huruf D | Dennis |
| '%a_' | Huruf Kedua dari Belakang a | Nicholas |
| '_ _ _' | Pas 3 Char | Jay, Ben |



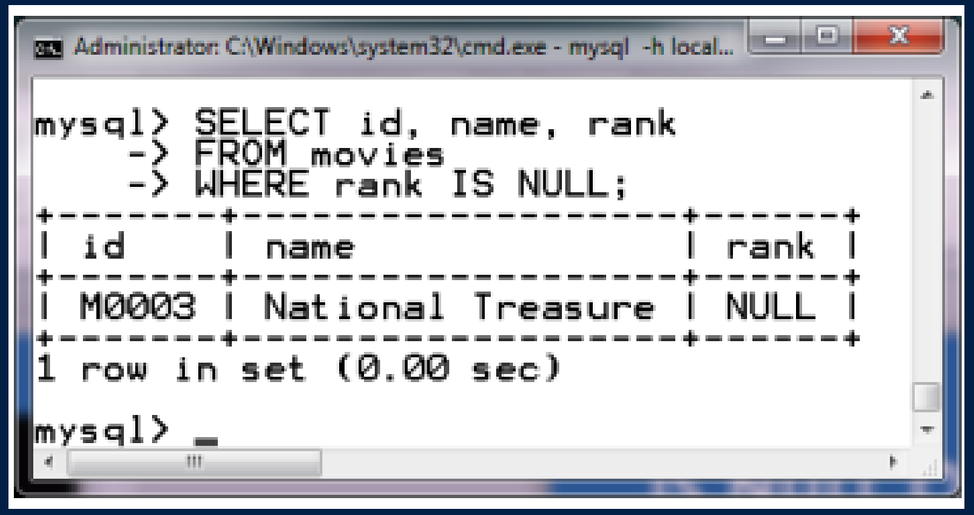



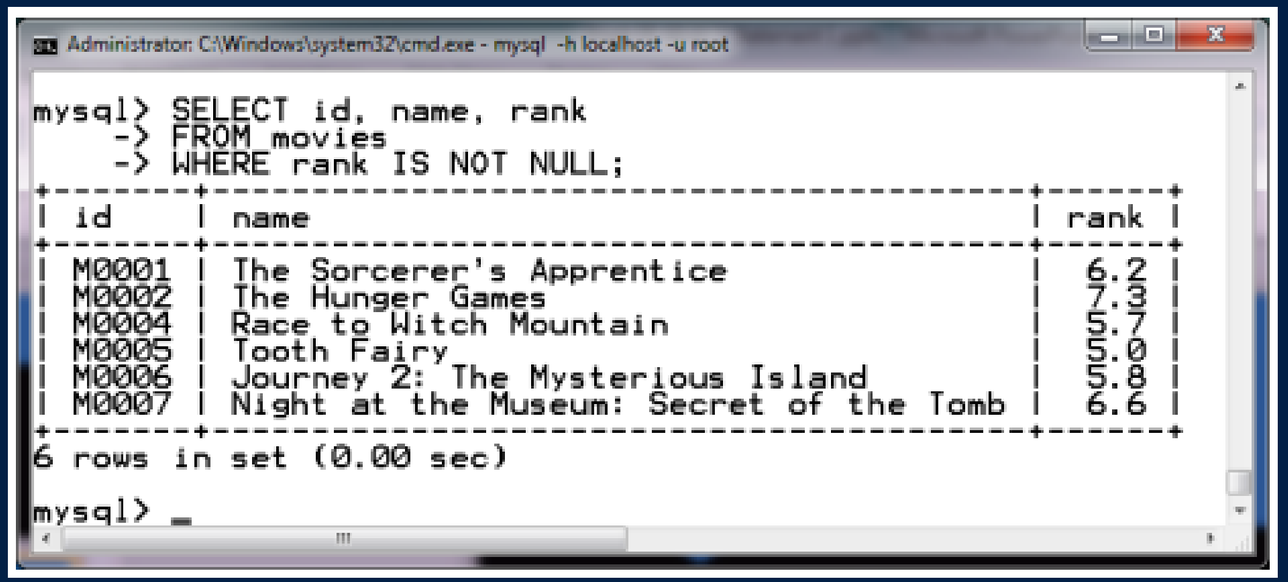
# IS NULL OPERATOR

Kita dapat menggunakan IS NULL Operator untuk mencari dan memeriksa NULL selain menggunakan “<=>”. Kita juga bisa menggunakan IS NOT NULL untuk mencari dan memeriksa nilai NOT NULL.

Syntax:

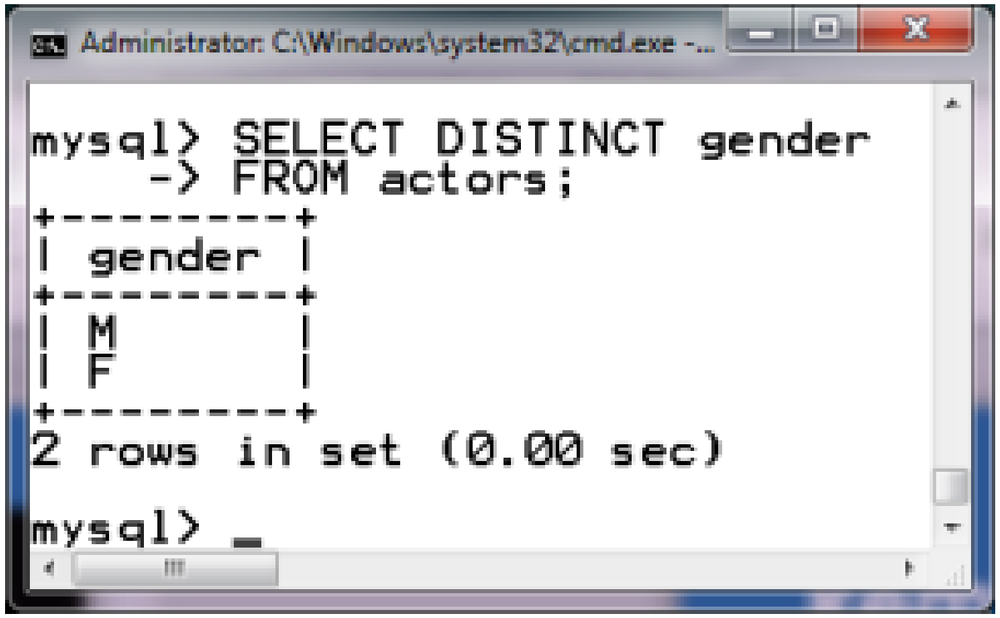
- WHERE x IS NULL
- WHERE x IS NOT NULL

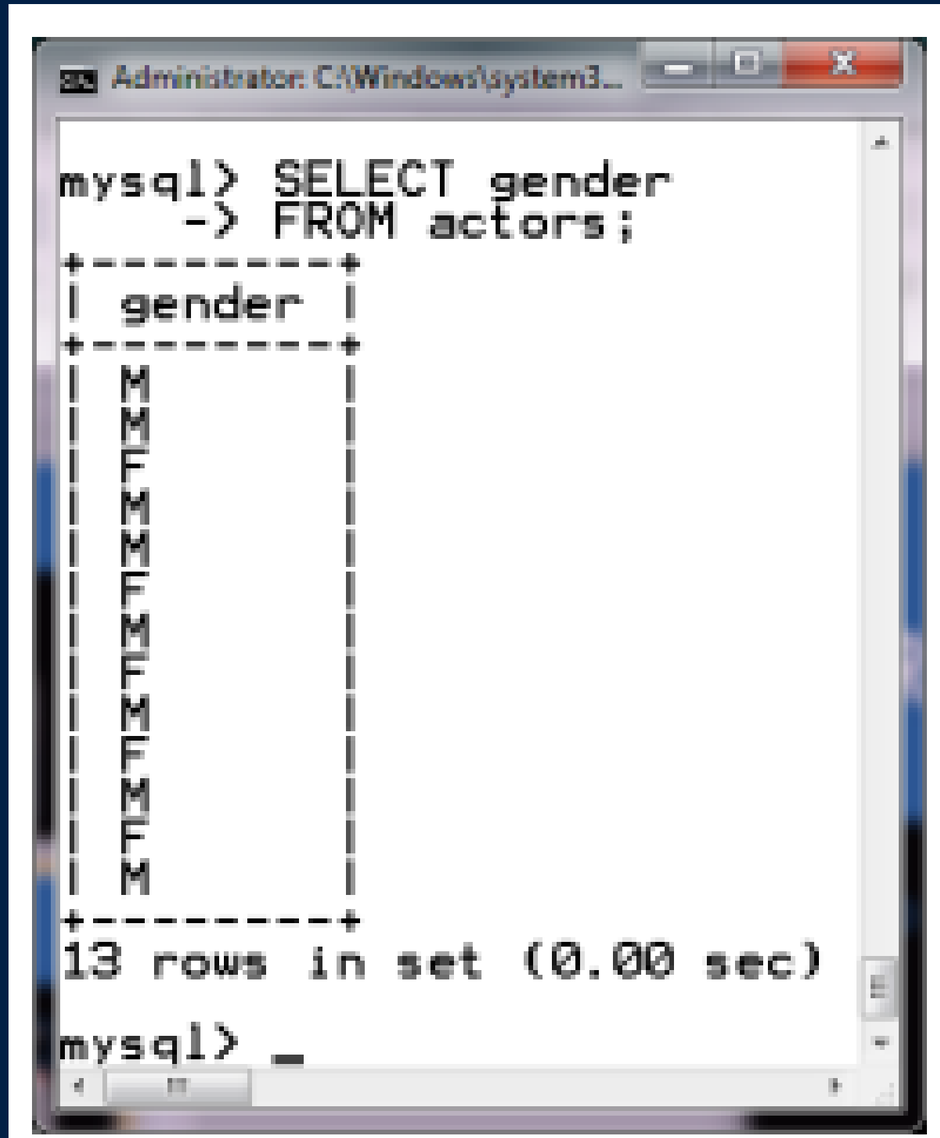


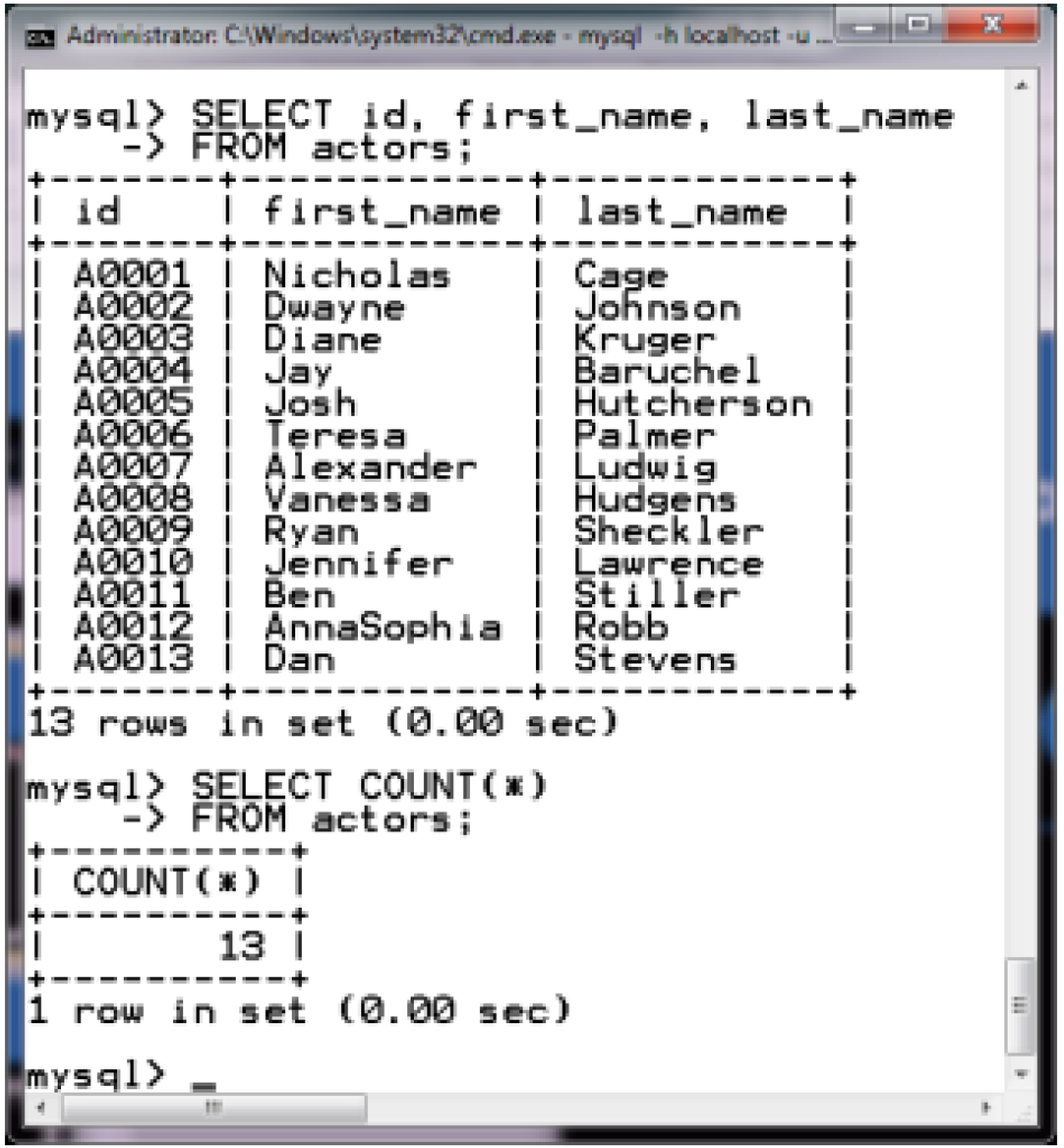
# DISTINCT OPERATOR

Kita dapat menggunakan DISTINCT Operator untuk mengeleminasi Duplicate Entry, terutama saat kita memerlukan nilai Unique.



# AGGR FUNC: COUNT FUNC

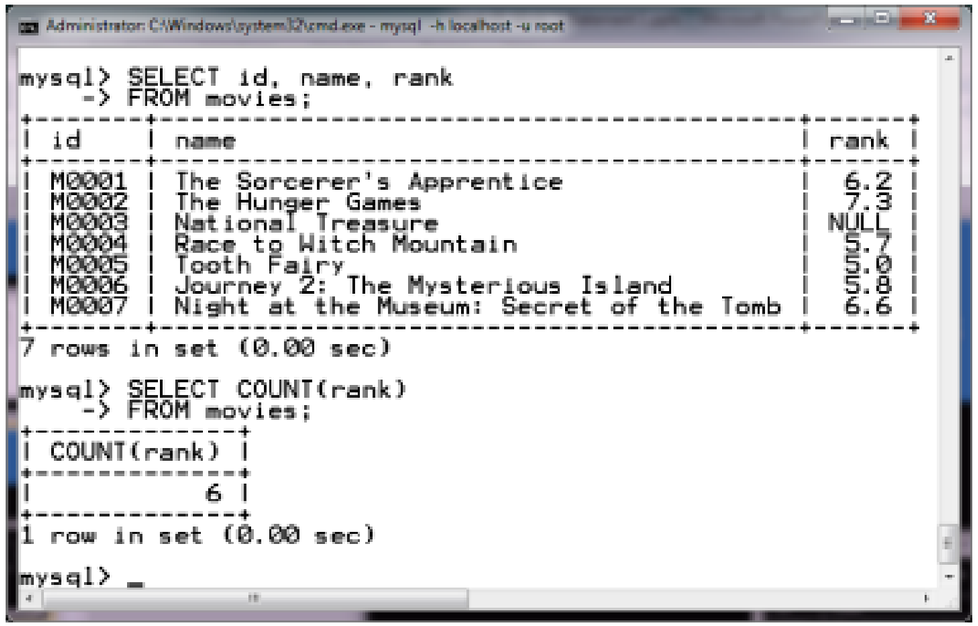


```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u ...

mysql> SELECT id, first_name, last_name
    -> FROM actors;
+-------+------------+------------+
| id    | first_name | last_name  |
+-------+------------+------------+
| A0001 | Nicholas   | Cage       |
| A0002 | Dwayne     | Johnson    |
| A0003 | Diane      | Kruger     |
| A0004 | Jay        | Baruchel   |
| A0005 | Josh       | Hutcherson |
| A0006 | Teresa     | Palmer     |
| A0007 | Alexander  | Ludwig     |
| A0008 | Vanessa    | Hudgens    |
| A0009 | Ryan       | Sheckler   |
| A0010 | Jennifer   | Lawrence   |
| A0011 | Ben        | Stiller    |
| A0012 | AnnaSophia | Robb       |
| A0013 | Dan        | Stevens    |
+-------+------------+------------+
13 rows in set (0.00 sec)

mysql> SELECT COUNT(*)
    -> FROM actors;
+----------+
| COUNT(*) |
+----------+
|       13 |
+----------+
1 row in set (0.00 sec)

mysql> _
```

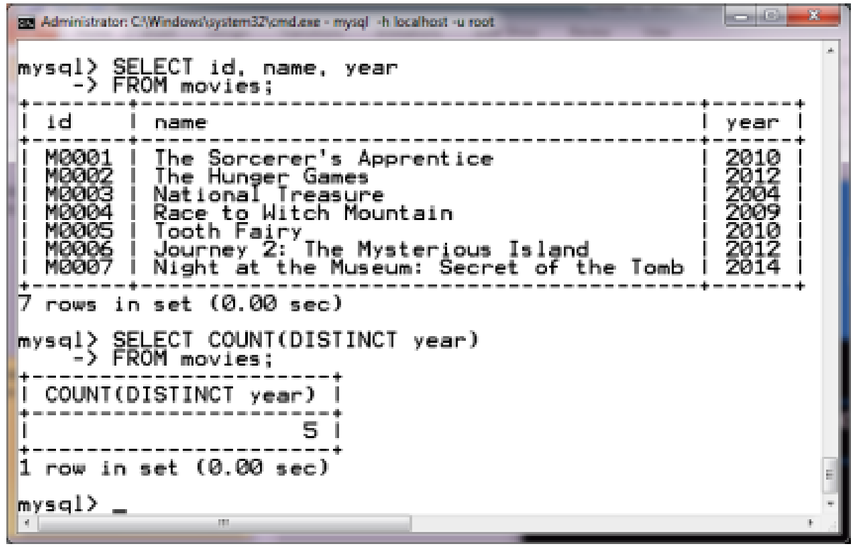


```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT id, name, rank
    -> FROM movies;
+-------+-----------------------------------------+------+
| id    | name                                    | rank |
+-------+-----------------------------------------+------+
| M0001 | The Sorcerer's Apprentice               |  6.2 |
| M0002 | The Hunger Games                        |  7.3 |
| M0003 | National Treasure                       | NULL |
| M0004 | Race to Witch Mountain                  |  5.7 |
| M0005 | Tooth Fairy                             |  5.0 |
| M0006 | Journey 2: The Mysterious Island        |  5.8 |
| M0007 | Night at the Museum: Secret of the Tomb |  6.6 |
+-------+-----------------------------------------+------+
7 rows in set (0.00 sec)

mysql> SELECT COUNT(rank)
    -> FROM movies;
+-------------+
| COUNT(rank) |
+-------------+
|           6 |
+-------------+
1 row in set (0.00 sec)

mysql> _
```



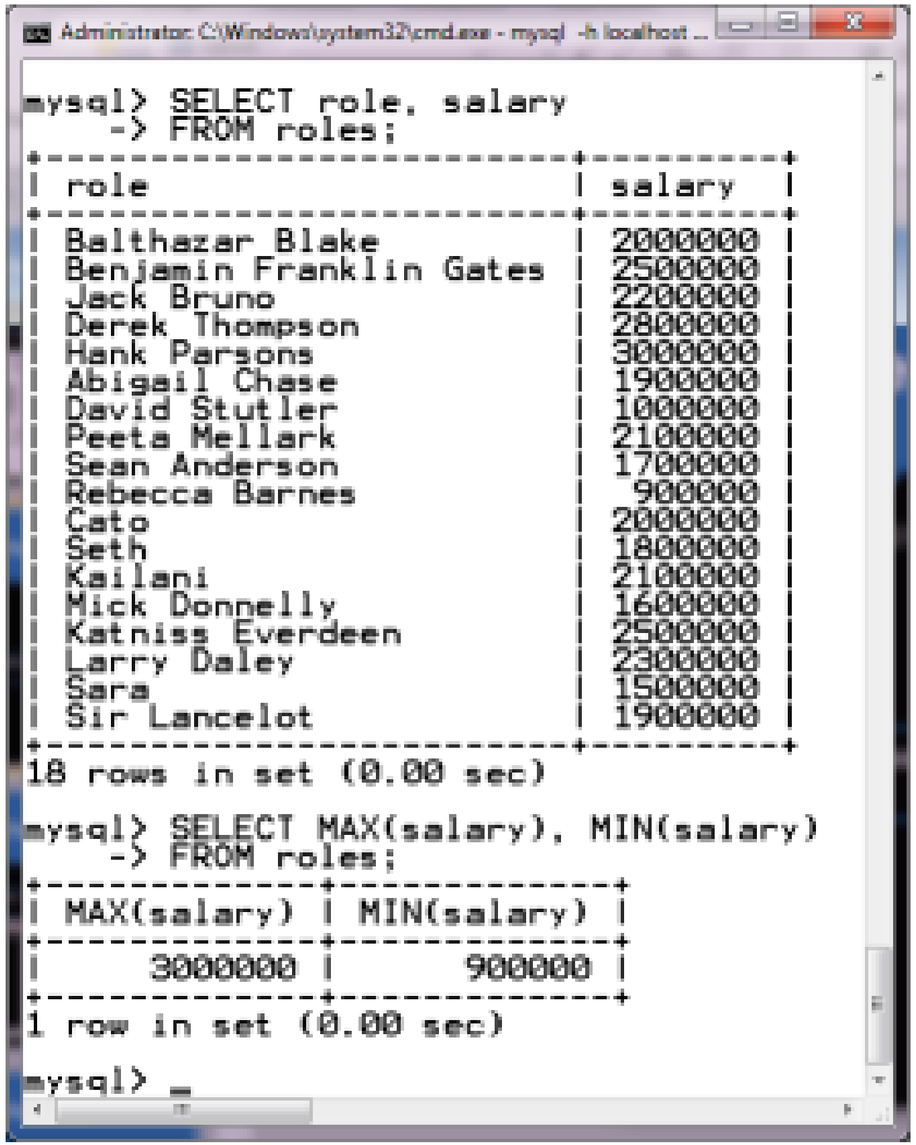
```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT id, name, year
    -> FROM movies;
+-------+-----------------------------------------+------+
| id    | name                                    | year |
+-------+-----------------------------------------+------+
| M0001 | The Sorcerer's Apprentice               | 2010 |
| M0002 | The Hunger Games                        | 2012 |
| M0003 | National Treasure                       | 2004 |
| M0004 | Race to Witch Mountain                  | 2009 |
| M0005 | Tooth Fairy                             | 2010 |
| M0006 | Journey 2: The Mysterious Island        | 2012 |
| M0007 | Night at the Museum: Secret of the Tomb | 2014 |
+-------+-----------------------------------------+------+
7 rows in set (0.00 sec)

mysql> SELECT COUNT(DISTINCT year)
    -> FROM movies;
+----------------------+
| COUNT(DISTINCT year) |
+----------------------+
|                    5 |
+----------------------+
1 row in set (0.00 sec)

mysql> _
```

# AGGR FUNC: MAX MIN FUNC

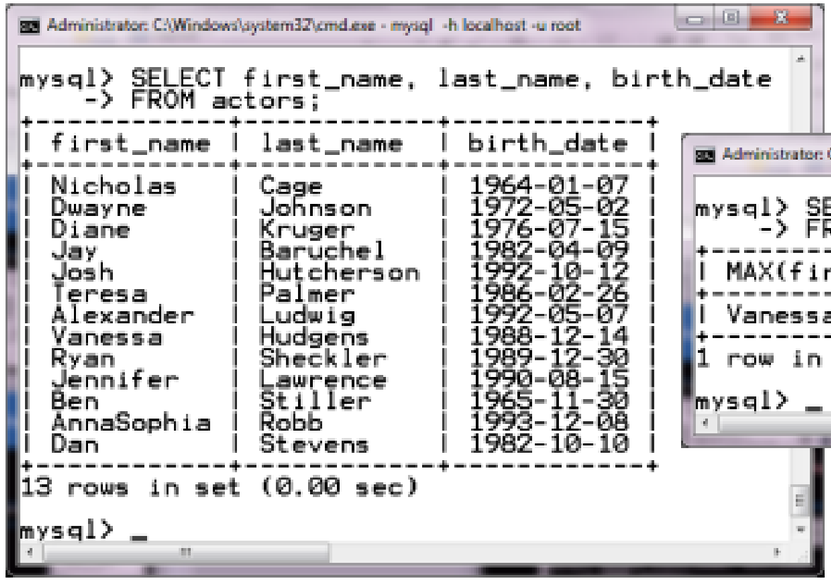


```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost ...

mysql> SELECT role, salary
    -> FROM roles;
+---------------------------+----------+
| role                      | salary   |
+---------------------------+----------+
| Balthazar Blake           |  2000000 |
| Benjamin Franklin Gates   |  2500000 |
| Jack Bruno                |  2200000 |
| Derek Thompson            |  2800000 |
| Hank Parsons              |  3000000 |
| Abigail Chase             |  1900000 |
| David Stutler             |  1000000 |
| Peeta Mellark             |  2100000 |
| Sean Anderson             |  1700000 |
| Rebecca Barnes            |   900000 |
| Cato                      |  2000000 |
| Seth                      |  1800000 |
| Kailani                   |  2100000 |
| Mick Donnelly             |  1600000 |
| Katniss Everdeen          |  2500000 |
| Larry Daley               |  2300000 |
| Sara                      |  1500000 |
| Sir Lancelot              |  1900000 |
+---------------------------+----------+
18 rows in set (0.00 sec)

mysql> SELECT MAX(salary), MIN(salary)
    -> FROM roles;
+-------------+-------------+
| MAX(salary) | MIN(salary) |
+-------------+-------------+
|     3000000 |      900000 |
+-------------+-------------+
1 row in set (0.00 sec)

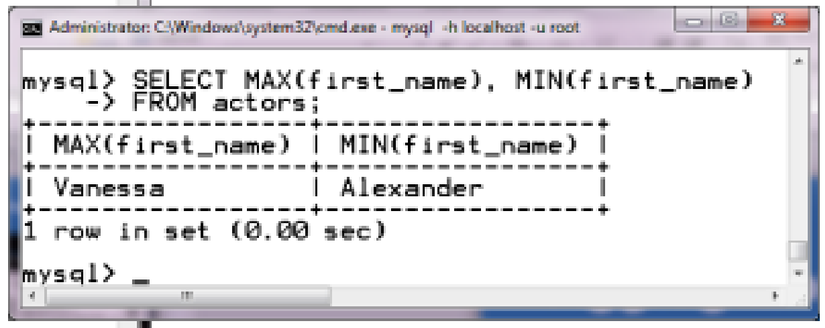
mysql> _
```



```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT first_name, last_name, birth_date
    -> FROM actors;
+------------+------------+------------+
| first_name | last_name  | birth_date |
+------------+------------+------------+
| Nicholas   | Cage       | 1964-01-07 |
| Dwayne     | Johnson    | 1972-05-02 |
| Diane      | Kruger     | 1976-07-15 |
| Jay        | Baruchel   | 1982-04-09 |
| Josh       | Hutcherson | 1992-10-12 |
| Teresa     | Palmer     | 1986-02-26 |
| Alexander  | Ludwig     | 1992-05-07 |
| Vanessa    | Hudgens    | 1988-12-14 |
| Ryan       | Sheckler   | 1989-12-30 |
| Jennifer   | Lawrence   | 1990-08-15 |
| Ben        | Stiller    | 1965-11-30 |
| AnnaSophia | Robb       | 1993-12-08 |
| Dan        | Stevens    | 1982-10-10 |
+------------+------------+------------+
13 rows in set (0.00 sec)

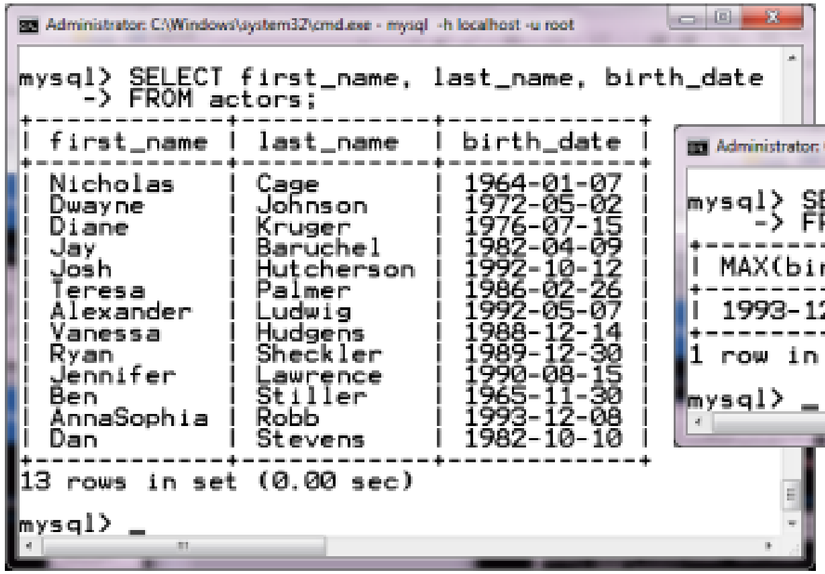
mysql> _
```



```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT MAX(first_name), MIN(first_name)
    -> FROM actors;
+------------------+------------------+
| MAX(first_name)  | MIN(first_name)  |
+------------------+------------------+
| Vanessa          | Alexander        |
+------------------+------------------+
1 row in set (0.00 sec)

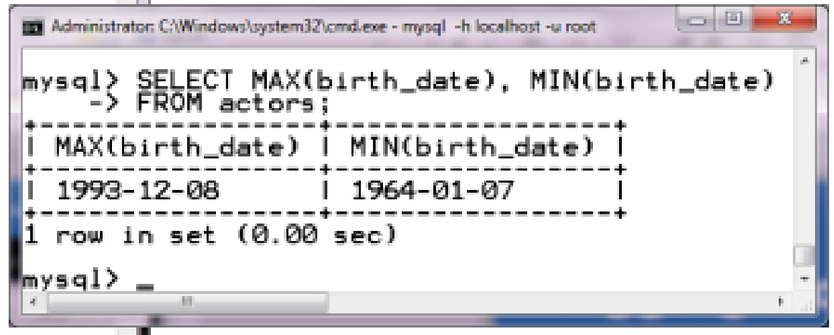
mysql> _
```



```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT first_name, last_name, birth_date
    -> FROM actors;
+------------+------------+------------+
| first_name | last_name  | birth_date |
+------------+------------+------------+
| Nicholas   | Cage       | 1964-01-07 |
| Dwayne     | Johnson    | 1972-05-02 |
| Diane      | Kruger     | 1976-07-15 |
| Jay        | Baruchel   | 1982-04-09 |
| Josh       | Hutcherson | 1992-10-12 |
| Teresa     | Palmer     | 1986-02-26 |
| Alexander  | Ludwig     | 1992-05-07 |
| Vanessa    | Hudgens    | 1988-12-14 |
| Ryan       | Sheckler   | 1989-12-30 |
| Jennifer   | Lawrence   | 1990-08-15 |
| Ben        | Stiller    | 1965-11-30 |
| AnnaSophia | Robb       | 1993-12-08 |
| Dan        | Stevens    | 1982-10-10 |
+------------+------------+------------+
13 rows in set (0.00 sec)

mysql> _
```



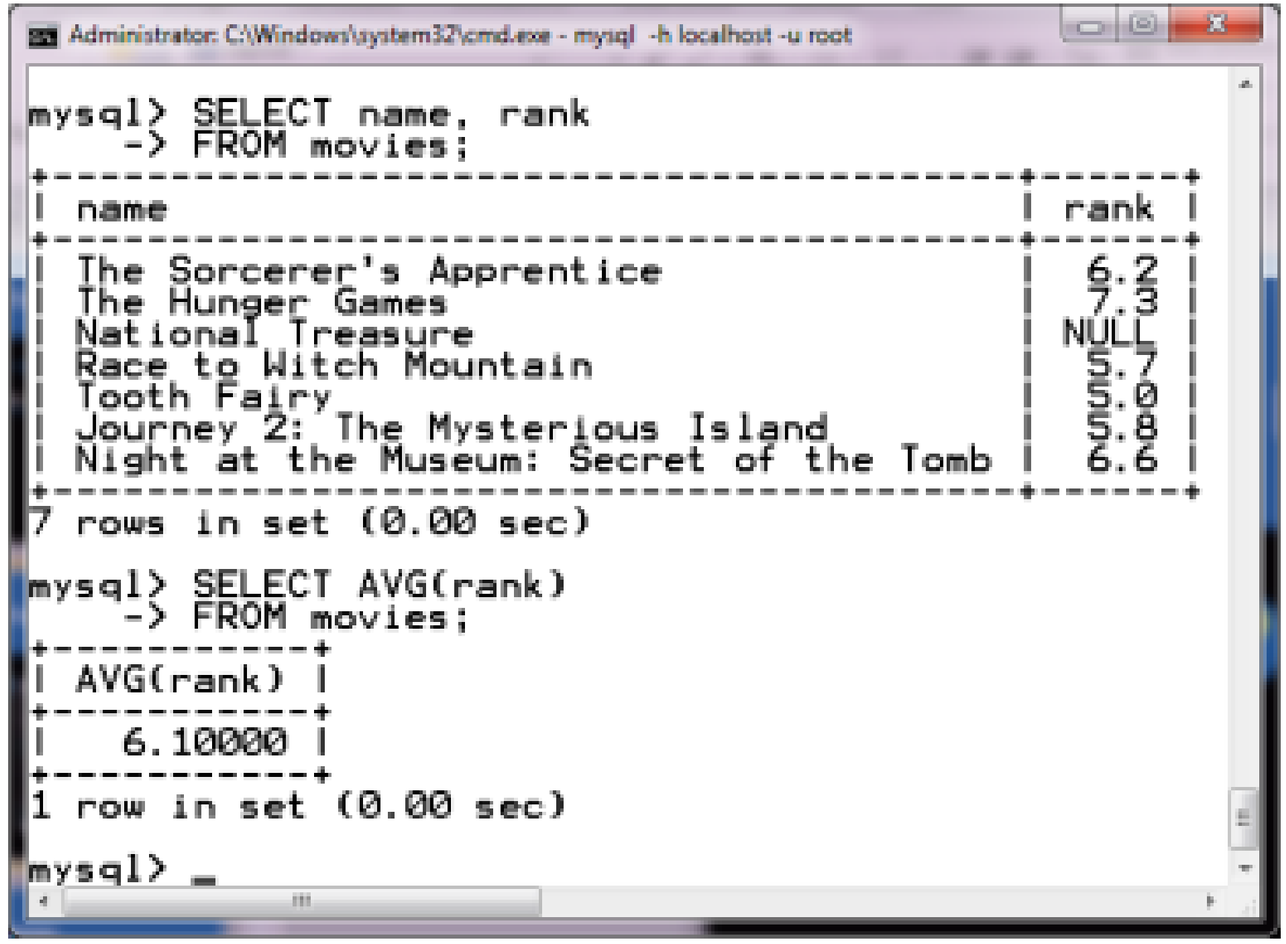
```
Administrator: C:\Windows\system32\cmd.exe - mysql  -h localhost -u root

mysql> SELECT MAX(birth_date), MIN(birth_date)
    -> FROM actors;
+-----------------+-----------------+
| MAX(birth_date) | MIN(birth_date) |
+-----------------+-----------------+
| 1993-12-08      | 1964-01-07      |
+-----------------+-----------------+
1 row in set (0.00 sec)

mysql> _
```

# AGGR FUNC: AVG & SUM FUNC



# TUGAS LAB IF351
# MEETING 06

# TERIMA KASIH